

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

R MANVI JAIN (1WA24CS226)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)

BENGALURU-560019
Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by R MANVI JAIN (1WA24CS226), who is a Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer

Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl No	Experiment Title	Page No
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-35
2	Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	36-44
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate-Monotonic b) Earliest-Deadline First c) Proportional Scheduling	45-61
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem	62-75
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection	76-94
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-Fit b) Best-Fit c) First-Fit	95-105
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	106-119

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating systems.

Program -1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS

b) SJF

c) Priority

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
```

```
int n, i;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
int at[n], bt[n], ct[n], tat[n],
```

```
wt[n]; printf("Enter Arrival
```

```
Time:\n"); for(i = 0; i < n; i++)
```

```
scanf("%d", &at[i]);
```

```
printf("Enter Burst
```

```
Time:\n"); for(i = 0; i < n;
```

```
i++) scanf("%d", &bt[i]);
```

```
int t = 0; for(i = 0; i < n; i++)
```

```
{ if(t <
```

```
at[i]) t =
```

```
at[i];
```

```
ct[i] = t + bt[i];
```

```
t = ct[i];
```

```

        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    return 0;
}

```

Result:

```

Enter total number of processes: 2

Process 1
Arrival Time: 0
Burst Time: 1

Process 2
Arrival Time: 0
Burst Time: 5

PID    AT    BT    CT    TAT    WT
P1      0     1     1     1     0
P2      0     5     6     6     1

Process returned 0 (0x0)   execution time : 11.955 s
Press any key to continue.
|

```

SJF (Non – Preemptive)

CODE:

```

#include <stdio.h>

int main()
{
    printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
    int n, i;
}

```

```

printf("Enter number of processes:
"); scanf("%d", &n); int at[n], bt[n],
ct[n], tat[n], wt[n]; int done[n];
printf("Enter Arrival Time:\n"); for(i
= 0; i < n; i++)
{
    scanf("%d", &at[i]);
    done[i] = 0;
}
printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
    scanf("%d", &bt[i]);
int t = 0, completed = 0;
while(completed < n)
{
    int min = 9999;
    int index = -1;
    for(i = 0; i < n;
i++)
    {
        if(at[i] <= t && done[i] == 0)
        {
            if(bt[i] < min)
            {
                min = bt[i];
                index = i;
            }
        }
    }

    if(index == -1)
    {

```

```

        t++;
    }
    else
    {
        ct[index] = t + bt[index];
        t = ct[index];

        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];

        done[index] = 1;
        completed++;
    }
}
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}
Result:

```

```

Enter total number of processes: 2

Process 1
Arrival Time: 0
Burst Time: 1

Process 2
Arrival Time: 0
Burst Time: 5

PID      AT      BT      CT      TAT      WT
P1        0        1        1        1        0
P2        0        5        6        6        1

Process returned 0 (0x0)   execution time : 11.955 s
Press any key to continue.
|

```

SRTF

CODE:

```

#include <stdio.h>

int main()
{
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
int n, i;
printf("Enter number of processes: ");
scanf("%d", &n);
int at[n], bt[n], rt[n], ct[n], tat[n],
wt[n]; printf("Enter Arrival Time:\n");
for(i = 0; i < n; i++)
scanf("%d", &at[i]);
printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
{

```

```

scanf("%d", &bt[i]);
rt[i] = bt[i];
}
int t = 0, completed = 0;
while(completed < n)
{
    int min = 9999;
    int index = -1;
    for(i = 0; i < n;
i++)
    {
        if(at[i] <= t && rt[i] > 0)
        {
            if(rt[i] < min)
            {
                min = rt[i];
                index = i;
            }
        }
    }
    if(index == -1)
    {
        t++;
    }
    else
    {
        rt[index]--; t++;
        if(rt[index] ==
0)
        {
            ct[index] = t; completed++;
            tat[index] = ct[index] - at[index];

```

```

        wt[index] = tat[index] -
        bt[index];
    }
}
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
return 0;
}

```

Result:

```

Enter number of processes: 4
Enter arrival times:
0
3
2
5
Enter burst times:
2
4
2
3

SJF Scheduling:
PID      AT      BT      CT      TAT      WT
P1        0        2        2        2        0
P2        3        4        8        5        1
P3        2        2        4        2        0
P4        5        3       11        6        3

Average Turnaround Time: 3.75
Average Waiting Time: 1.00
BY Raghav N 1wa24cs227
Process returned 0 (0x0)   execution time : 13.619 s
Press any key to continue.

```

PRIORITY NON-PREEMPTIVE

CODE:

```

#include <stdio.h>

int main()
{
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
int n, i;

printf("Enter number of processes: ");
scanf("%d", &n);

int at[n], bt[n], pr[n], ct[n], tat[n],
wt[n]; int done[n]; printf("Enter
Arrival Time:\n"); for(i = 0; i < n; i++)

```

```

{
    scanf("%d", &at[i]);
    done[i] = 0;
}
printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
    scanf("%d", &bt[i]);

printf("Enter Priority:\n");
for(i = 0; i < n; i++)
    scanf("%d", &pr[i]);

int t = 0, completed = 0;

while(completed < n)
{
    int min = 9999;
    int index = -1;
    for(i = 0; i < n;
    i++)
    {
        if(at[i] <= t && done[i] == 0)
        {
            if(pr[i] < min)
            {
                min = pr[i];
                index = i;
            }
        }
    }

    if(index == -1)

```

```

    {
        t++;
    }
    else
    {
        ct[index] = t + bt[index]; t =
        ct[index]; tat[index] = ct[index] -
        at[index]; wt[index] = tat[index]
        - bt[index];

        done[index] = 1;
        completed++;
    }
}

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

return 0;
}
Result:

```

```

Enter number of processes: 5
Enter Arrival Time, Burst Time, Priority for P1: 0 4 2
Enter Arrival Time, Burst Time, Priority for P2: 1 3 3
Enter Arrival Time, Burst Time, Priority for P3: 2 1 4
Enter Arrival Time, Burst Time, Priority for P4: 3 5 5
Enter Arrival Time, Burst Time, Priority for P5: 4 2 2

--- Priority Non-Preemptive ---
Process AT      BT      PR      CT      TAT      WT
P1      0      4      2      4      4      0
P2      1      3      3      9      8      5
P3      2      1      4     10      8      7
P4      3      5      5     15     12      7
P5      4      2      2      6      2      0
Average TAT = 6.80
Average WT = 3.80
raghav N- 1wa24cs227
Process returned 0 (0x0)   execution time : 79.590 s
Press any key to continue.

```

PRIORITY PREEMPTIVE

CODE:

```

#include <stdio.h>

int main()
{
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], rt[n];
    int ct[n], tat[n], wt[n];

```

```

printf("Enter Arrival Time:\n");
for(i = 0; i < n; i++)
    scanf("%d", &at[i]);

```

```

printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
{
    scanf("%d", &bt[i]);
    rt[i] = bt[i];
}

```

```

printf("Enter Priority:\n");
for(i = 0; i < n; i++)
    scanf("%d", &pr[i]);

```

```

int t = 0, completed = 0;
while(completed < n)
{
    int min = 9999;
    int index = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= t && rt[i] > 0)
        {
            if(pr[i] < min)
            {
                min = pr[i];
                index = i;
            }
        }
    }
}

```

```

    if(index == -1)
    {
        t++;
    }
else
{
    rt[index]--;
    t++;

    if(rt[index] == 0)
    {
        ct[index] = t; tat[index] =
        ct[index] - at[index]; wt[index] =
        tat[index] - bt[index];
        completed++;
    }
}
}

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

```

Enter number of processes: 5
Enter Arrival Time, Burst Time, Priority for P1: 0 3 5
Enter Arrival Time, Burst Time, Priority for P2: 2 2 3
Enter Arrival Time, Burst Time, Priority for P3: 3 5 2
Enter Arrival Time, Burst Time, Priority for P4: 4 4 4
Enter Arrival Time, Burst Time, Priority for P5: 6 1 1

--- Priority Preemptive ---
Process AT      BT      PR      CT      TAT      WT
P1      0       3       5      15      15      12
P2      2       2       3      10       8       6
P3      3       5       2       9       6       1
P4      4       4       4      14      10       6
P5      6       1       1       7       1       0
Average TAT = 8.00
Average WT = 5.00
raghav n- 1wa24cs227
Process returned 0 (0x0)   execution time : 68.490 s
Press any key to continue.

```

ROUND ROBIN

CODE:

```

#include <stdio.h>

int main()
{
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
    int n, tq, i;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

```

```

int pid[n], at[n], bt[n], rt[n];
int wt[n], tat[n], ct[n], resp[n];
int queue[100]; int front=0,
rear=0;

for(i=0;i<n;i++)
{
    printf("Enter Process ID, Arrival Time, Burst Time:\n");
    scanf("%d %d %d",&pid[i],&at[i],&bt[i]);

    rt[i] = bt[i];
    wt[i] = 0;
    tat[i] = 0;
    resp[i] = -1;
}
int current_time = 0;
int completed = 0;
int visited[n];

for(i=0;i<n;i++)
    visited[i] = 0;

while(completed < n)
{
    for(i=0;i<n;i++)
    {
        if(at[i] <= current_time && visited[i] == 0)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
}

```

```

if(front == rear)
{
    current_time++;
    continue;
}

int p = queue[front++];

if(resp[p] == -1)
    resp[p] = current_time - at[p];

if(rt[p] > tq)
{
    current_time += tq;
    rt[p] -= tq;

    for(i=0;i<n;i++)
    {
        if(at[i] <= current_time && visited[i] == 0)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    queue[rear++] = p;
}
else
{
    current_time += rt[p];
    rt[p] = 0;
}

```

```

        ct[p] = current_time;
        tat[p] = ct[p] - at[p];
        wt[p] = tat[p] - bt[p];
        completed++;
    }
}

float avg_wt=0, avg_tat=0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i],resp[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);

return 0;
}

```

Result:

```

Enter number of processes: 5
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 5
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 3
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 1
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 2
Enter Arrival Time for P5: 4
Enter Burst Time for P5: 3
Enter Time Quantum: 2

--- Round Robin Scheduling ---
Process AT      BT      CT      TAT      WT
P1      0       5      14      14       9
P2      1       3      12      11       8
P3      2       1       5       3       2
P4      3       2       7       4       2
P5      4       3      13       9       6
Average TAT = 8.20
Average WT = 5.40
raghav n- 1wa24cs227
Process returned 0 (0x0)   execution time : 57.287 s
Press any key to continue.

```

Program -2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
```

```
int n, i;
```

```

printf("Enter number of processes: ");
scanf("%d", &n);

int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], q[n];

printf("Enter Arrival Time:\n");
for(i = 0; i < n; i++)
scanf("%d", &at[i]);

printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
{
    scanf("%d", &bt[i]);
    rt[i] = bt[i];
}

printf("Enter Queue (1 = RR, 2 = FCFS):\n");
for(i = 0; i < n; i++)
    scanf("%d", &q[i]);

int tq;
printf("Enter Time Quantum:
"); scanf("%d", &tq); int t = 0,
completed = 0; printf("\nGantt
Chart:\n| "); while(completed <
n)
{
    int found = 0;
    for(i = 0; i < n;
i++)
    {
        if(q[i] == 1 && rt[i] > 0 && at[i] <= t)

```

```

{
    int exec = (rt[i] > tq) ? tq : rt[i];
    for(int k = 0; k < exec; k++)
    {
        printf("P%d ", i+1);
        t++;
    }
    rt[i] -= exec;
    if(rt[i] == 0)
    { ct[i] =
        t;
        completed++;
    }

    found = 1;
}
}
if(!found)
{
    for(i = 0; i < n; i++)
    {
        if(q[i] == 2 && rt[i] > 0 && at[i] <= t)
        {
            while(rt[i] > 0)
            {
                // check if any Q1 arrives →
                break int interrupt = 0; for(int j =
                0; j < n; j++)
                {
                    if(q[j] == 1 && rt[j] > 0 && at[j] <= t)
                    {

```

```

        interrupt = 1;
        break;
    }
}
if(interrupt) break;
printf("P%d ", i+1);
rt[i]--;
t++;
}
if(rt[i] == 0)
{ ct[i] =
  t;
  completed++;
}
found = 1;
}
}
}
// CPU idle
if(!found)
{ printf("Idle ");
  t++;
}
}
printf("\n");
// Calculate TAT & WT
for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

```

```
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}
```

Result:

```

Enter number of processes: 4
Enter Arrival Time:
0
0
0
10
Enter Burst Time:
4
3
8
5
Enter Queue Number (1-System RR, 2-User FCFS):
1 1 2 1
Enter Time Quantum: 2

Process Q   AT   BT   CT   TAT  WT
P1  1    0    4    6    6    2
P2  1    0    3    7    7    4
P3  2    0    8   15   15    7
P4  1   10    5   20   10    5

Average Turnaround Time = 9.50
Average Waiting Time = 4.50

Raghav N 1wa24cs227

```

Program -3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)

- Rate- Monotonic
- Earliest-deadline First
- Proportional scheduling

RATE – MONOTONIC

CODE:

```

#include <stdio.h>
#include<math.h>

int main()
{
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
    int n, i, j, t = 0, lcm = 1;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int burst[n], period[n], remaining[n];

    printf("Enter CPU burst times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &burst[i]);

    printf("Enter time periods:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &period[i]);

    for(i = 0; i < n; i++)
    {
        int max = (lcm > period[i]) ? lcm : period[i];
        while(1)
        {
            if(max % lcm == 0 && max % period[i] == 0)
            {
                lcm = max;
                break;
            }
        }
    }
}

```

```

        max++;
    }
}
printf("\nLCM=%d\n", lcm);
printf("\nRate Monotonic Scheduling:\n");
printf("PID\tBurst\tPeriod\n"); for(i = 0; i
< n; i++)
    printf("%d\t%d\t%d\n", i+1, burst[i], period[i]);
float util = 0; for(i
= 0; i < n; i++)
    util += (float)burst[i] / period[i];

float bound = n * (pow(2, (float)1/n) - 1);

printf("\n%f <= %f => %s\n", util, bound,
    ( util <= bound) ? "true" : "false");

for(i = 0; i < n; i++)
    remaining[i] = 0;

printf("\nScheduling occurs for %d ms\n", lcm);

for(t = 0; t < lcm; t++)
{

    for(i = 0; i < n; i++)
    {
        if(t % period[i] == 0)
            remaining[i] = burst[i];
    }
}

```

```

    int min = -1; for(i
= 0; i < n; i++)
{
    if(remaining[i] > 0)
    {
        if(min == -1 || period[i] < period[min])
            min = i;
    }
}

if(min != -1)
{
    printf("%dms onwards: Process %d running\n", t, min+1); remaining[min]-
-;
}
else
{
    printf("%dms onwards: CPU is idle\n", t);
}
}
return 0;
}

```

Result:

```

Rate Monotone Scheduling:
PID    Burst    Period
1       20       50
2       35       100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle
76ms onwards: CPU is idle
77ms onwards: CPU is idle
78ms onwards: CPU is idle
79ms onwards: CPU is idle
80ms onwards: CPU is idle
81ms onwards: CPU is idle
82ms onwards: CPU is idle
83ms onwards: CPU is idle
84ms onwards: CPU is idle
85ms onwards: CPU is idle
86ms onwards: CPU is idle
87ms onwards: CPU is idle
88ms onwards: CPU is idle
89ms onwards: CPU is idle
90ms onwards: CPU is idle

```

EARLIEST DEADLINE FIRST

CODE:

```

#include <stdio.h>
#include <limits.h>

```

```

int main() {
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");

int n, i, time, hyperperiod = 20; printf("Enter number
of tasks: "); scanf("%d", &n); int C[n], D[n], T[n];
int remaining[n]; int next_arrival[n]; int
abs_deadline[n];

for(i = 0; i < n; i++) {
    printf("Enter Capacity, Deadline, Period for Task %d: ", i +
    1); scanf("%d %d %d", &C[i], &D[i], &T[i]); remaining[i] =
    0; next_arrival[i] = 0; abs_deadline[i] = 0;
}
printf("\n--- EDF Scheduling (Periodic Tasks) ---
\n");

for(time = 0; time < hyperperiod; time++) {
    for(i = 0; i < n; i++) {
        if(time == next_arrival[i]) {
            remaining[i] = C[i];
            abs_deadline[i] = time + D[i];
            next_arrival[i] += T[i];
        }
    }
    int selected = -1;
    int min_deadline = INT_MAX;
    for(i = 0; i < n; i++) {
        if(remaining[i] > 0 && abs_deadline[i] < min_deadline) {
            min_deadline = abs_deadline[i];
            selected = i;
        }
    }
}
}

```

```
    if(selected == -1) {  
        printf("%d ms : CPU is idle.\n", time);  
    } else { printf("%d ms : Task %d is running.\n", time,  
        selected + 1); remaining[selected]--;  
    }  
}  
  
return 0;  
}
```

Result:

```

Enter number of tasks: 3
Enter Burst times:
3
2
2
Enter Deadlines:
7
4
8
Enter Periods:
20
5
10

PID      Burst  Deadline  Period
1         3       7         20
2         2       4          5
3         2       8         10

Scheduling occurs for 20 ms

0ms onwards: Task 2 is running.
1ms onwards: Task 2 is running.
2ms onwards: Task 1 is running.
3ms onwards: Task 1 is running.
4ms onwards: Task 1 is running.
5ms onwards: Task 2 is running.
6ms onwards: Task 2 is running.
7ms onwards: Task 3 is running.
8ms onwards: Task 3 is running.
9ms onwards: CPU is idle.
10ms onwards: Task 2 is running.
11ms onwards: Task 2 is running.
12ms onwards: Task 3 is running.
13ms onwards: Task 3 is running.
14ms onwards: CPU is idle.
15ms onwards: Task 2 is running.
16ms onwards: Task 2 is running.
17ms onwards: CPU is idle.
Raghav N-1wa24cs227
Process returned 0 (0x0)   execution time : 38.017 s
Press any key to continue.

```

PROPORTIONAL SHARE SCHEDULING

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {

```

```

    char
name[5]; int
tickets; }
Process;

int main() {
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
int n, total_tickets = 0; float total_T = 0.0;

printf("Enter the number of Processes:
"); scanf("%d", &n); Process p[n];
srand(time(NULL));

for (int i = 0; i < n; i++) {
    printf("\nProcess %d:\n", i +
1); sprintf(p[i].name, "P%d", i
+ 1); printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T += p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);

for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets +
1; int accumulated_tickets = 0; int
winner_index;

    for (int j = 0; j < n; j++) {

```

```

    accumulated_tickets += p[j].tickets; if
(winning_ticket <= accumulated_tickets) {
    winner_index = j;
    break;
}
}

printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
    p[winner_index].name);
}

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
        ((p[i].tickets / total_T) * 100));
}

return 0;
}
Result:

```

```

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 35, Winner: P3
Tickets picked: 35, Winner: P3
Tickets picked: 9, Winner: P1
Tickets picked: 37, Winner: P3
Tickets picked: 59, Winner: P3

The Process: P1 gets 16.67% of Processor Time.
raghav n -lwa24cs227
The Process: P2 gets 33.33% of Processor Time.
raghav n -lwa24cs227
The Process: P3 gets 50.00% of Processor Time.
raghav n -lwa24cs227
Process returned 0 (0x0)    execution time : 27.021 s
Press any key to continue.

```

Program -4

Question:

Write a C program to simulate: (Any one)

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

PRODUCER – CONSUMER

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int top = -1;
int item = 0;

sem_t empty, full, mutex;

void produce() {
    sem_wait(&empty);
    sem_wait(&mutex);

    item++; top++;
    buffer[top] = item;

    printf("Producer has produced: Item %d\n", item);
    sem_post(&mutex); sem_post(&full);
}

void consume() {
    sem_wait(&full);
    sem_wait(&mutex);

    int consumed = buffer[top];
    printf("Consumer has consumed: Item %d\n", consumed);

    top--;

```

```

    sem_post(&mutex);
    sem_post(&empty);
}

int main() {
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
    int choice;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    while (1) {
        printf("\nEnter 1. Producer 2. Consumer 3.
        Exit\n"); printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (top == BUFFER_SIZE - 1) {
                    printf("Buffer is full!\n");
                } else {
                    produce();
                }
                break;
            case 2:
                if (top == -1) {
                    printf("Buffer is empty!\n");
                } else {
                    consume();
                }
                break;

```

```
        case 3:
            printf("Exiting...\n");
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
}

return 0;
}
```

Result:

```
Enter the size of the queue:
3

--- Producer Consumer Menu ---
1. Produce (Insert Item)
2. Consume (Remove Item)
3. Quit
Enter your choice: 1
Enter item to produce: 10
Produced item: 10

--- Producer Consumer Menu ---
1. Produce (Insert Item)
2. Consume (Remove Item)
3. Quit
Enter your choice: 1
Enter item to produce: 56
Produced item: 56

--- Producer Consumer Menu ---
1. Produce (Insert Item)
2. Consume (Remove Item)
3. Quit
Enter your choice: 1
Enter item to produce: 58
Produced item: 58

--- Producer Consumer Menu ---
1. Produce (Insert Item)
2. Consume (Remove Item)
3. Quit
Enter your choice: 1
Enter item to produce: 52
Queue Overflow! Cannot insert item.
```

DINING – PHILOSOPHER’S

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id; int right =
    (id + 1) % N;

    for (int i = 0; i < 1; i++) {
        printf("Philosopher %d is thinking.\n",
            id); sleep(1); if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked left fork %d\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id, right);
        } else {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id, right);

            pthread_mutex_lock(&forks[left]); // FIXED TYPO HERE printf("Philosopher %d
            picked left fork %d\n", id, left);
```

```

    }

    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d\n", id, left, right);
}

return NULL;
}

int main() {
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
int ids[N];

for (int i = 0; i < N; i++) {
    pthread_mutex_init(&forks[i], NULL);
    ids[i] = i;
}

for (int i = 0; i < N; i++) {
    pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
}

for (int i = 0; i < N; i++) {
    pthread_join(philosophers[i], NULL);
}

```

```

    }

    for (int i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}

```

Result:

```

Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 2 picked left fork 2
Philosopher 2 picked right fork 3
Philosopher 2 is eating.
Philosopher 0 picked left fork 0
Philosopher 0 picked right fork 1
Philosopher 0 is eating.
Philosopher 3 picked right fork 4
Philosopher 0 put down forks 0 and 1
Philosopher 1 picked right fork 2
Philosopher 1 picked left fork 1
Philosopher 1 is eating.
Philosopher 2 put down forks 2 and 3
Philosopher 3 picked left fork 3
Philosopher 3 is eating.
Philosopher 1 put down forks 1 and 2
Philosopher 3 put down forks 3 and 4
Philosopher 4 picked left fork 4
Philosopher 4 picked right fork 0
Philosopher 4 is eating.
Philosopher 4 put down forks 4 and 0

...Program finished with exit code 0
Press ENTER to exit console.

```

Program -5

Question:

Write a C program to simulate: (Any one)

a) Bankers' algorithm for the purpose of deadlock avoidance.

b) Deadlock Detection

BANKER'S ALGORITHM

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
int n, m, i, j, k;
printf("Enter number of processes: ");
scanf("%d", &n); printf("Enter number
of resources: "); scanf("%d", &m); int
alloc[n][m], max[n][m], need[n][m];
int avail[m];
printf("Enter Allocation Matrix:\n");
for(i = 0; i < n; i++)
{
for(j = 0; j < m; j++)
{
scanf("%d", &alloc[i][j]);
}
}
printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++)
{
for(j = 0; j < m; j++)
{ scanf("%d", &max[i][j]);
}
}
}
```

```

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{
    scanf("%d", &avail[i]);
}
// Calculate Need Matrix
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
int finish[n],
safeSeq[n]; int
work[m]; for(i = 0; i <
n; i++) finish[i] = 0;
for(i = 0; i < m; i++)
    work[i] = avail[i];
int count = 0;
while(count < n)
{
    int found = 0;
    for(i = 0; i < n;
i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;
            for(j = 0; j < m; j++)
            {
                if(need[i][j] > work[j])

```

```

        {
            possible = 0;
            break;
        }
    }
    if(possible)
    {
        for(k = 0; k < m; k++)
        {
            work[k] += alloc[i][k];
        }
        safeSeq[count] =
        i; count++;
        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
{
    printf("System is not in a safe state.\n");
    return 0;
}

printf("\nSystem is in a safe
state.\n"); printf("Safe sequence is:
"); for(i = 0; i < n; i++)
{
    printf("P%d", safeSeq[i]);
    if(i != n - 1)
        printf(" -> ");
}

```

```

    printf("\n");
    return 0;
}

```

Result:

```

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
raghav N-1wa24cs227

```

DEADLOCK DETECTION

CODE:

```

#include <stdio.h>

int main()
{
    printf("Name: MANVI N\nUSN:1WA24CS227\n\n");
    int n, m, i, j, k; printf("Enter number of processes:
    "); scanf("%d", &n); printf("Enter number of
    resources: "); scanf("%d", &m); int alloc[n][m],

```

```

req[n][m]; int avail[m], finish[n], safe[n];
printf("Enter allocation matrix:\n"); for(i = 0; i < n;
i++)
{
    printf("Process %d: ", i);
    for(j = 0; j < m; j++)
        scanf("%d", &alloc[i][j]);
}
printf("Enter request matrix:\n");
for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);
    for(j = 0; j < m; j++)
        scanf("%d", &req[i][j]);
}
printf("Enter available resources:
"); for(i = 0; i < m; i++)
scanf("%d", &avail[i]); for(i = 0; i
< n; i++)

    finish[i] = 0;
int count = 0;
while(count < n)
{
    int found = 0;
    for(i = 0; i < n;
i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;
            for(j = 0; j < m; j++)

```

```

    {
        if(req[i][j] > avail[j])
        {
            possible = 0;
            break;
        }
    }
    if(possible)
    {
        for(k = 0; k < m; k++)
            avail[k] += alloc[i][k];
        safe[count] = i;
        count++;
        finish[i] = 1;
        found = 1;
    }
}
}
if(found == 0)
    break;
}
if(count == n)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
        printf("P%d ", safe[i]);
}
else
{

```

```

        printf("\nDeadlock detected.\n");
    }

    return 0;
}

```

Result:

```

Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0
System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

```

Program -6

Question:

Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-fit

b) Best-fit

c) First-fit

CODE:

```
#include <stdio.h>
```

```
void firstFit(int blocks[], int m, int processes[], int n) {
```

```

int alloc[n], used[m];
for (int i = 0; i < m; i++)
used[i] = 0;
for (int i = 0; i < n; i++)
    alloc[i] = -1;

for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        if (!used[j] && blocks[j] >= processes[i])
            { alloc[i] = j; used[j] = 1; break;
            }

printf("\n--- First Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void bestFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

```

```

for (int i = 0; i < n; i++)
{
    int best = -1;
    for (int j = 0; j < m; j++)
        if (!used[j] && blocks[j] >= processes[i])
            if (best == -1 || blocks[j] < blocks[best])
                best = j;
    if (best != -1) {
        alloc[i] = best;
        used[best] = 1;
    }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else
        printf("Not Allocated\n");
}

void worstFit(int blocks[], int m, int processes[], int n)
{
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;
}

```

```

for (int i = 0; i < n; i++)
{
    int worst = -1;
    for (int j = 0; j < m; j++)
        if (!used[j] && blocks[j] >= processes[i])
            if (worst == -1 || blocks[j] > blocks[worst])
                worst = j;
    if (worst != -1) {
        alloc[i] = worst;
        used[worst] = 1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process\n\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else
        printf("Not Allocated\n");
}

int main() {

printf("Name: MANVI N\nUSN:1WA24CS227\n\n");

int m, n;

printf("Enter number of memory blocks:");
scanf("%d", &m);
int blocks[m];
printf("Enter sizes of %d memory blocks:\n", m);
for (int i = 0; i < m; i++)

```

```

scanf("%d", &blocks[i]);

printf("Enter number of processes:
"); scanf("%d", &n); int
processes[n];
printf("Enter sizes of %d processes:\n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &processes[i]);

firstFit(blocks, m, processes, n);
bestFit(blocks, m, processes, n);
worstFit(blocks, m, processes, n);

return 0;
}

```

Result:

```

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.      Process Size      Block No.
1                212                2
2                417                5
3                112                3
4                426                Not Allocated

--- Best Fit ---
Process No.      Process Size      Block No.
1                212                4
2                417                2
3                112                3
4                426                5

--- Worst Fit ---
Process No.      Process Size      Block No.
1                212                5
2                417                2
3                112                4
4                426                Not Allocated

...Program finished with exit code 0
Press ENTER to exit console.

```

Program -7

Question:

Write a C program to simulate page replacement algorithms. (Any one)

- a) FIFO
- b) LRU
- c) Optimal

FIFO

CODE:

```

#include <stdio.h>

int main() {

    printf("Name: MANVI N\nUSN: 1WA24CS227\n\n");

    int pages[20], frames[10]; int n,
    f, i, j, k = 0, fault = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string: ");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++) {
        frames[i] = -1;
    }
    for(i = 0; i < n; i++) {

        found = 0;

        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
    }
}

```

```

    }

    if(found == 0) {
        frames[k] = pages[i];
        k = (k + 1) % f;
        fault++;
    }

    printf("\nPage %d -> ", pages[i]);

    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
}

printf("\n\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Result:

```

Enter number of pages: 12
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 2

Page 7 -> 7 -
Page 0 -> 7 0
Page 1 -> 1 0
Page 2 -> 1 2
Page 0 -> 0 2
Page 3 -> 0 3
Page 0 -> 0 3
Page 4 -> 4 3
Page 2 -> 4 2
Page 3 -> 3 2
Page 0 -> 3 0
Page 3 -> 3 0

Total Page Faults = 10

...Program finished with exit code 0
Press ENTER to exit console.

```

LRU

CODE:

```

#include <stdio.h>

int main() {
    printf("Name: MANVI N\nUSN: 1WA24CS227\n\n");
    int pages[20], frames[10], time[10]; int n, f, i, j, pos,
    fault = 0, count = 0, found; printf("Enter number of
    pages: "); scanf("%d", &n); printf("Enter page
    reference string: "); for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames:
    "); scanf("%d", &f); for(i = 0; i <
    f; i++) {

```

```

    frames[i] = -1;
}
for(i = 0; i < n; i++) {
    found = 0; for(j = 0;
j < f; j++) {
        if(frames[j] == pages[i]) {
            count++;
            time[j] = count;
            found = 1;
            break;
        }
    }
    if(found == 0) {
        pos = 0; for(j = 1; j
< f; j++) {
            if(time[j] < time[pos]) {
                pos = j;
            }
        }
        frames[pos] =
pages[i]; count++;
        time[pos] = count;
        fault++;
    }
    printf("\nPage %d -> ", pages[i]);
    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
}

```

```

printf("\n\nTotal Page Faults = %d\n", fault);
return 0;
}

```

Result:

```

Enter number of pages: 12
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page 1 -> - 1 -
Page 2 -> - 2 -
Page 3 -> - 3 -
Page 4 -> - 4 -
Page 1 -> - 1 -
Page 2 -> - 2 -
Page 5 -> - 5 -
Page 1 -> - 1 -
Page 2 -> - 2 -
Page 3 -> - 3 -
Page 4 -> - 4 -
Page 5 -> - 5 -

Total Page Faults = 12

...Program finished with exit code 0
Press ENTER to exit console.

```

OPTIMAL PAGE REPLACEMENT

CODE:

```

#include <stdio.h>

int main() {
    printf("Name: MANVI N\nUSN: 1WA24CS227\n\n");
    int pages[20], frames[10]; int n, f, i, j, k, pos, fault = 0,
    found; printf("Enter number of pages: "); scanf("%d",
    &n); printf("Enter page reference string: "); for(i = 0; i
    < n; i++) {
        scanf("%d", &pages[i]);
    }
}

```

```

}
printf("Enter number of frames:
"); scanf("%d", &f); for(i = 0; i <
f; i++) {
    frames[i] = -1;
}
for(i = 0; i < n; i++) {
    found = 0; for(j = 0;
j < f; j++) {
        if(frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }
}
if(found == 0) {
    pos = -1; for(j = 0; j
< f; j++) {
        int future = 0;
        for(k = i + 1; k < n; k++) {
            if(frames[j] == pages[k]) {
                future = 1;
                break;
            }
        }
        if(future == 0) {
            pos = j;
            break;
        }
    }
}
if(pos == -1)
    pos = 0;

```

```

        frames[pos] = pages[i];
        fault++;
    }
    printf("\nPage %d -> ", pages[i]);
    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
}
printf("\n\nTotal Page Faults = %d\n", fault);
return 0;
}

```

Result:

```

Enter number of pages: 12
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 1 2 3
Page 2 -> 1 2 3
Page 5 -> 5 2 3
Page 1 -> 1 2 3
Page 2 -> 1 2 3
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 5 -> 5 2 3

Total Page Faults = 9

```